

Project 3. Cache Coherence

Due 23:59, Tuesday, June 9th, 2026 (KST)

TA: Guseul Heo (gsheo@casys.kaist.ac.kr)

1. Introduction

On the surface, cache coherence seems straightforward; all caches simply must see all operations on a piece of data in the same order. Implementation of coherence, however, is not so simple. In this project, you will be creating a simulator that maintains coherent caches for 4, 8, and 16 core SMP. You will be implementing the MSI, MESI, and MOESI protocols for a bus-based broadcast system.

Please read the descriptions below carefully and complete the missing components of the given framework. If you have any questions, feel free to ask the TA via Piazza.

2. Specification of Simulator

A simulator will be provided that is capable of simulating 4, 8, and 16-core CMP systems. Each core will have a memory trace reader, which reads the trace files provided to you.

Each core in the CMP has one level of cache. The cache has a single cycle lookup time. The base cache code is provided for you and is used to track coherence states for accessed blocks; you do not need to implement cache replacement or eviction. You will only need to implement the protocol files that process cache and snoop requests. The CMP has a single memory controller which can access the off-chip memory. This memory controller is provided for you and will respond to any query (GETS or GETM) placed on the bus with data after a 100-cycle delay.

The bus modeled is an atomic bus. This means that once a query is placed on the bus, the bus will not allow any other requests onto the bus until a DATA response is seen. Caches request the bus using the `bus_request()` function. If the bus is not available, it will place the request on an arbitration queue to be scheduled in the future. This is done on a first-come, first-served basis, with node 0 having the highest priority and node N having the lowest priority.

Each processor (trace reader) will have up to one outstanding memory request at a time. The processor will send a request to the cache and will wait until the cache responds with a DATA message. (This will be done using the `send_DATA_to_proc()` command)

We have provided a full simulator framework in C++. This framework creates the simulator, reads in the traces, creates a basic cache structure, and creates the memory controller. All of this code is in the `sim/` directory of the downloadable code. **You should not need to change ANY code in the `sim/` directory.**

The `protocols/` directory contains files you may need to modify. Most notably, you need to implement the protocol files. When a request comes from a processor to the cache, the cache finds the entry and then calls `process_cache_request()` in the protocol. In this function, you should look at the cache entry's state and decide what messages (if any) should be sent, and what state the cache should transition to. When a request is snooped on the bus, the cache finds the entry and then calls `process_snoop_request()` in the protocol.

To help you in understanding the framework, the `MI` protocol is already completed and given to you. You MUST fill in the following files:

`MSI_protocol.h/.cpp` , `MESI_protocol.h/.cpp` , `MOESI_protocol.h/.cpp` .

In order to interface properly with the simulator, **do not change any of the class names or delete any functions. You may however need to add additional functions, states, and/or messages** in order to complete the assignment.

Important Notes About Simulator Assumptions:

1. All requests that are not DATA (GETS and GETM) always expect to have someone reply with DATA. To ensure this, the memory will always respond 100 cycles after the request with DATA unless another cache places DATA on the bus first.

In traditional protocols, there are certain messages that do not expect replies (e.g. Bus_Upgrade). These types of messages are not supported because every non-DATA request expects a data response (e.g. GETS and GETM). This creates situations where the cache sending the GETS or GETM may be the one that should supply the data. In these cases, the cache should simply send DATA to itself on the bus. Given the difficulties in identifying your own GETS or GETM request, the validations assume that you only reply when you are already responsible for supplying the data.

2. In general, there is more than one way to implement each protocol. For this project, the reference implementations were made with simple logic and emphasizing cache to cache transfers, while running in minimal time. For example, contrary to the MESIF article in Wikipedia, the reference implementation will keep the F state at the same processor and not move it.

3. Simulator Options

Build

```
$ make
```

Basic command

```
$ ./sim_trace -t [TRACE_DIRECTORY] -p [PROTOCOL]
```

Options

- `t` : Directory with the trace you want to run
- `p` : Protocol you want to run, the options are: `MI` , `MSI` , `MESI` , `MOESI`

Tips

We provide output files that may be helpful for your development. The output files show the simulator logs and the statistics that your simulator should generate for each cache coherence protocol. If your simulator produces identical results to the output files, it is implemented correctly.

4. Outputs / Statistics

The framework prints detailed simulator logs and final cache coherence states for debugging. For grading, we will compare the following final statistics:

1. Number of cycles to complete execution (Already output by framework)
2. Number of cache misses (This can be due to a cold miss or coherence)
3. Number of cache accesses (Already output by framework)
4. Number of "silent upgrades" for the MESI protocol
5. Number of Cache-to-cache transfers (This refers to the number of times data is not supplied by Memory)

5. Hints

- A suggested starting point is to create a framework where you can read the traces, look up data in the caches, and use the MI protocol.
- Look up information on cache coherence and state diagrams for assistance.
- **Start early: This is a difficult project.**

6. Grading Policy

Your code will be evaluated using three public traces provided to you, and will be graded based on its performance with both the public traces and hidden traces that

have not been announced.

Your grade will be determined based on the following criteria:

- **100%:** Produces the same statistics as the correct answer for all traces.
- **95%:** Produces the same statistics as the correct answer for public traces, but not for the hidden traces.
- **85%:** Produces a result different from the correct answer, but with an average error of under **5%**.
- **50%:** Produces a result different from the correct answer, with an average error of over **5%**.
- **0%:** Code is not compilable or does not produce any result.

Only the statistics described in [4. Outputs / Statistics] will be graded. Logs and final cache-state dumps in the middle or at the end of the simulation will not be graded.

7. Submission

Zip your `protocols` directory and name it with **your student id**.

```
$ tar -cvf [student id].tar protocols
```

Submit the tar file to KLMS.

Please make sure that your code compiles and runs well. We will give 0 points to the code that does not compile without any exception.

gcc / g++ version = 13.3.0

8. Late Policy

You will lose **30%** of your score on the **first day** (June 10th 00:00 ~ 23:59). We will **not accept** any works that are submitted after then.

If you have any requests or questions regarding administrative issues (such as late submission due to an unfortunate accident), please send an e-mail to TA.